

Modern Cloud Computing

Lecture 14: Cloud Application Performance Optimization

Chongzhou Fang

Rochester Institute of Technology

Today's Topics

- Basics
- Elasticity and Scaling
- Data
- Networking
- System Design
- Testing and Benchmarking

Slow Cloud Application

A slow cloud application may be slow because of:

- Inefficient application code
- Poor compute sizing
- Too much synchronous communication
- Expensive database queries
- Missing cache layer
- Network distance or jitter
- Cold starts or slow startup
- Bad scaling thresholds

Performance Efficiency

Goal: Use cloud resources efficiently while maintaining performance as demand and technologies change.

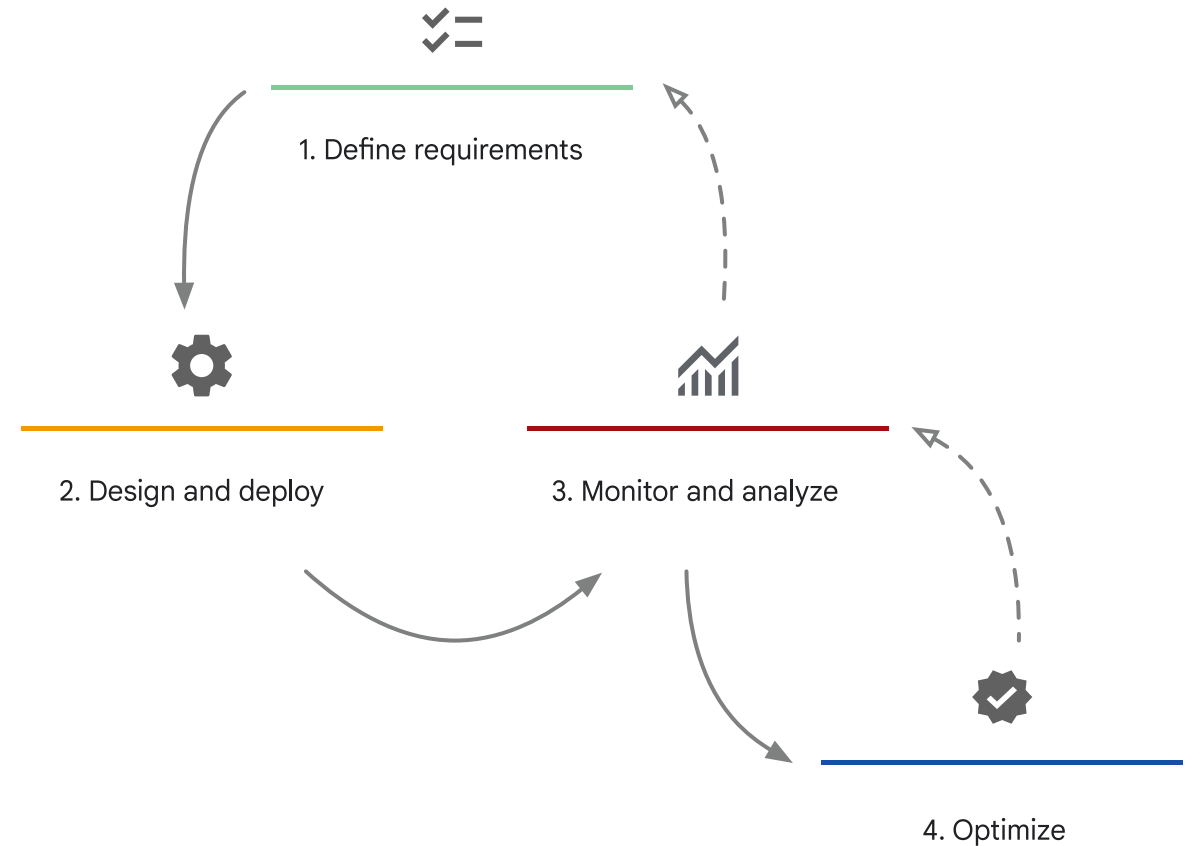
- Architecture selection
- Compute and hardware
- Data management
- Networking and content delivery
- Process and culture

Performance Optimization

A continuous cycle:

1. Define requirements
2. Design and deploy
3. Monitor and analyze
4. Optimize

The cycle repeats as the workload evolves.



Design Principles

AWS states five design principles for performance efficiency:

- **Democratize advanced technologies:** Make advanced technology implementation easier for your team by delegating complex tasks to your cloud vendor. Consume specialized technology (NoSQL, transcoding, ML) as managed services instead of building the expertise in-house
- **Go global in minutes:** deploy in multiple regions to lower latency for users
- **Use serverless architectures:** remove the burden of running servers; managed services operate at cloud scale
- **Experiment more often:** virtual, automatable resources make comparative testing of instance types, storage, and configurations cheap
- **Consider mechanical sympathy:** pick the technology that aligns with how the workload actually behaves, e.g. select a data store from its access pattern

Optimization Loop

A practical loop for cloud performance:

Requirements

- Architecture choices
- Implementation
- Load testing
- Production monitoring
- Bottleneck analysis
- Optimization
- Re-test and repeat

Performance Metrics

Metric	Description
Average latency	General user experience
p95 / p99 latency	Tail behavior under load
Throughput	Work completed per unit time
Error rate	Whether the system is failing under pressure
Saturation	How close a resource is to its limit
Queue length	Whether demand exceeds service capacity
Cost per request	Efficiency of the architecture

Tail Latency

Average latency may not work well.

Example:

```
Average latency: 120 ms  
p95 latency:    700 ms  
p99 latency:    2.4 s
```

In distributed systems, a request often depends on many sub-requests. The slowest component can dominate the user-visible response time.

Start with the Workload

Different workloads need different optimization strategies.

Workload	Performance concern
Web API	latency, autoscaling
Batch analytics	throughput, data locality, parallelism
Streaming pipeline	backpressure, queueing, end-to-end delay
ML inference	accelerator choice, model latency, concurrency
Interactive UI	network distance, caching, rendering time
Transaction system	consistency, database contention

Architecture Selection

Optimal solution varies by workload.

Cloud provides multiple architectural choices:

- Virtual machines for control and compatibility
- Containers for packaging and portability
- Serverless for elastic event-driven tasks
- Specialized hardware for acceleration

Make the choice data-driven. Benchmark candidate services and configurations instead of choosing from guesses, assumptions, or on-premises habits.

Trade-Offs

Performance optimization is full of trade-offs.

Optimization	Possible trade-off
Add cache	stale data, invalidation complexity
Add replicas	consistency and coordination cost
Split services	network latency and debugging cost
Use serverless	cold starts and platform limits
Use accelerators	portability and utilization concerns
Increase resources	higher cost and possible waste

Right-Sizing Compute

Right-sizing means matching resources to actual workload needs.

Under-provisioning causes:

- High latency
- Queue buildup
- Error spikes
- Poor customer experience

Over-provisioning causes:

- Low utilization
- Higher cost

Elasticity and Scaling

Elasticity

Elasticity: the ability to adjust resources dynamically based on workload changes, so that different components can scale independently.

- Establish a baseline load
- Define the response to increases **and** decreases
- Define what triggers scaling
- Pre-scale for known peaks; autoscale for real-time change

In practice this also needs:

- Clear scaling signals
- Appropriate thresholds
- Fast startup
- Stateless or externally stateful services
- Load balancing

Peaks

Traffic pattern	Better strategy
Known seasonal sale	pre-scale before event
Daily predictable cycle	scheduled or predictive scaling
Unexpected viral traffic	reactive autoscaling
Sporadic event-driven tasks	serverless or queue-based workers
Slow long-term growth	periodic rightsizing and capacity planning

Autoscaling Metrics

Common autoscaling signals:

- CPU utilization
- Memory usage
- Request rate
- Queue depth
- Load balancer serving capacity
- Latency
- Custom application metrics

Use the metric closest to the bottleneck.

AWS's test: the metric must describe how busy an instance is **and** change proportionally with the number of instances.

Example (AWS): a video-transcoding fleet runs at 100% CPU by design, so CPU is useless as a signal. Scale on the depth of the job queue instead.

Scaling Pitfalls

Autoscaling can fail when:

- Startup time is too slow
- Health checks are misconfigured
- Scaling threshold is too high
- Cooldown is too long (slow reaction) or too short (flapping)
- Quotas prevent more resources
- State is stored locally
- Traffic spikes faster than scaling response

Serverless and Elasticity

Serverless platforms can simplify elasticity. Google Cloud's point: serverless autoscaling is near-instant and can **scale to zero**, with no scaling rules to tune.

They are useful when workloads are:

- Event-driven
- Bursty
- Stateless
- Short-running
- Easy to split into independent tasks

But serverless can introduce:

- Cold-start latency
- Concurrency limits
- Timeout limits
- Observability challenges
- Platform-specific behavior

Data

Data Management

AWS emphasizes purpose-built data stores.

Data design depends on:

- Data type: block, file, object, relational, key-value, graph
- Access pattern: random vs sequential
- Throughput and IOPS requirements
- Read/write ratio
- Update frequency
- Durability and availability needs
- Query shape

Storage and Access Patterns

Access pattern	Common design choice
Hot key-value reads	cache or key-value store
Complex transactions	relational database
Large immutable objects	object storage
High-throughput analytics	data warehouse / columnar store
Search queries	search index
Time-series metrics	time-series database
Graph traversal	graph database

Caching

Caching reduces latency and backend load.

Common cache locations:

- Browser cache
- CDN edge cache
- API gateway cache
- Application cache
- Database query cache

Caching works best when data is read frequently and changes less frequently.

Challenges: invalidation and consistency.

Cache Design Considerations

- What exactly is cached?
- Where is the cache located?
- What is the TTL (Time to Live)?
- How is stale data handled?
- What happens on cache miss?
- Is the cache shared or local?

AWS guidance:

- **Client-side cache:** lowest latency, but duplicates data and can diverge between clients
- **Remote cache:** consistent and scales independently, but adds a network hop
- Combine both for multi-level caching
- Choose an invalidation strategy
- Target a cache hit rate of **80% or higher**

Networking

Networking and Content Delivery

Network performance depends on:

- Latency
- Throughput
- Jitter
- Bandwidth
- Packet loss
- User location
- Protocol choice

Reduce Network Distance

Latency often improves when users and services are closer.

Techniques:

- Choose regions close to users
- Use edge caching or CDN
- Use global load balancing
- Place dependent services close together
- Avoid cross-region synchronous calls
- Use private connectivity for hybrid workloads
- Keep large data near compute

Moving data is often more expensive than moving computation.

Load Balancing

Load balancing improves performance by spreading traffic across resources, and is what lets a workload actually use the elasticity of the cloud.

AWS guidance:

- Choose the balancer type from workload characteristics: protocol (TCP, HTTP, TLS, WebSockets), target type (instances, containers, serverless), long-lived connections or stickiness, and placement
- Offload TLS termination to the balancer
- Let the balancer use HTTP/2 or gRPC to clients even if the backends do not

But load balancing cannot fix a backend bottleneck that every request still depends on.

Protocol and Payload Optimization

Network optimization also involves application protocols.

Examples:

- Reduce payload size
- Compress responses when useful
- Use HTTP/2 or HTTP/3 where appropriate
- Batch small requests carefully
- Avoid chatty APIs
- Use asynchronous messaging for slow paths
- Use streaming for large responses

AWS frames this as transport choice:

- **TCP** for reliable delivery; TLS handshakes add round-trips
- **UDP** for fast, loss-tolerant traffic: logs, metrics, VoIP
- **gRPC over HTTP/2** for compact binary payloads between services

System Design

Modular Design

Google Cloud emphasizes modular design for flexible scaling and independent updates.

Modularity helps when components have different:

- Load profiles
- Scaling needs
- Availability requirements
- Data requirements
- Deployment frequency
- Security boundaries

But modularity adds communication overhead and latency; Google notes a **tightly coupled** design can be the better choice for performance-critical systems.

Loose Coupling

A loosely coupled architecture reduces direct dependencies.

Common mechanisms:

- APIs with stable contracts
- Message queues
- Pub/sub systems
- Event-driven workflows
- Service discovery
- Backpressure mechanisms
- Idempotent operations

Loose coupling improves resilience and scaling, but can complicate debugging and consistency.

Concurrency and Parallelism

Applications should be designed to perform multiple tasks concurrently when possible.

Examples:

- Serve many user requests at once
- Process independent jobs in parallel
- Split large tasks into smaller chunks
- Run background work outside request path
- Use worker pools and queues

Concurrency enables cloud resources to be used efficiently.

Stateless Models

Stateless application processes are easier to scale and recover.

Twelve-factor processes are **stateless and share-nothing**: each request is handled independently, and memory or local disk is only a single-transaction cache.

Persistent state should live in backing services:

- Database
- Object storage
- Distributed cache
- Queue
- Session store (a time-expiring store such as Redis or Memcached)

Twelve-factor is explicit: sticky sessions should **never** be used or relied upon.

Twelve-Factor App

The Twelve-Factor App methodology targets SaaS applications.

It emphasizes:

- Setup automation
- Portability across execution environments
- Deployment on modern cloud platforms
- Minimal divergence between development and production
- Scalability without major changes to tooling or architecture

Codebase and Dependencies

Twelve-factor applications use one codebase per app and many deploys.

They also explicitly declare and isolate dependencies.

Performance relevance:

- Reproducible builds
- Less environment drift
- Easier rollback
- Consistent performance testing
- Easier comparison across versions

A performance regression is easier to identify when deploys are reproducible.

Config and Backing Services

Store deploy-specific config in **environment variables**. Never as code constants.

Backing services (databases, queues, caches) are attached resources reached through a URL or credentials in that config; local and third-party services look the same to the code.

- Swap database/cache endpoints without code changes
- Change capacity by changing attached resources
- Test with realistic staging services
- Avoid hard-coded environment assumptions

Build, Release, Run

Twelve-factor apps strictly separate:

Codebase → Build → Release → Run

- Build: Converts code repo into an executable bundle (*build*).
- Release: Takes the *build* and combines it with the deploy's current config.
- Run: runs the app in the execution environment.

Benefits:

- Runtime is kept simple
- Releases are traceable
- Rollback is easier
- Tests can be tied to specific releases

Processes and Concurrency

Apps execute as stateless processes and scale out through the **process model** (stateless, share-nothing)

- Web processes for HTTP requests
- Worker processes for long-running background jobs

Processes rely on the platform's process manager for output streams, crash handling, and restarts.

Disposability

Disposability means fast startup and graceful shutdown: processes can be started or stopped at a moment's notice.

- Startup in a few seconds
- Graceful shutdown on `SIGTERM` signal from process manager:
 - A web process stops accepting connections, finishes in-flight requests, then exits
 - A worker returns its current job to the queue, so jobs must be reentrant (transactions or idempotent operations)
- Robustness against **sudden death** (hardware failure), via a queue backend that returns jobs on disconnect

Disposability: Benefits

- Faster autoscaling response
- Safer deploys
- Faster recovery after failure
- Easier platform scheduling
- Less disruption during rolling updates

A slow-starting service may fail under sudden load because capacity arrives too late.

Dev/Prod Parity

Development, staging, and production should be as similar as possible.

Three goals:

- Make **Time** gap small: should take only hours between writing and deploying
- Make **Personnel** gap small: the people who write code should also deploy and watch it
- Make **Tool** gap small: same type **and version** of each backing service in dev and prod

Benefits

- Avoid surprises after deployment
- Catch dependency and configuration issues earlier
- Make load tests more meaningful
- Reduce environment-specific tuning

Performance tests on unrealistic environments can be misleading.

Logs as Event Streams

Twelve-factor apps treat logs as event streams: the app writes its stream, unbuffered, to `stdout`, and the execution environment decides where it is routed and stored.

- App does not manage log files directly
- Platform can aggregate and analyze logs
- Logs can be correlated with metrics and traces
- Centralized logs support bottleneck analysis

Observability is not optional for performance work.

Admin Processes

Admin tasks (migrations or a console against the live database) run as **one-off processes**.

One-off admin processes should be run in an identical environment as the regular long-running processes of the app

- Same release, codebase, and config as the long-running processes
- Same dependency isolation
- Admin code ships with the application code

Testing and Benchmarking

Performance Testing

Performance tests should be automated and tied to builds.

Useful tests:

- Microbenchmarks for critical code paths
- Load tests for steady traffic
- Stress tests for breaking points
- Spike tests for sudden demand

Benchmarking vs Load Testing

Benchmark (synthetic, component-level)

- Quick to set up; runs early in the pipeline
- Evaluates one component, often before a full system exists
- Pre-warm; run several times to capture variance
- Custom tests for your operations; industry tests (TPC-DS) to compare environments

Load test (whole workload, production-like)

- Test the entire workload
- Production-like infrastructure
- Synthetic or sanitized production data
- Go **beyond** expected load to find where scaling stops or turns non-linear
- Automate in the pipeline; compare against predefined KPIs

Load Generation

Test scripts should be:

- Repeatable
- Idempotent where possible (same result whether it is applied once or multiple times)
- Not tightly coupled to one another
- Representative of production behavior

Load tests that do not resemble production behavior often optimize the wrong thing.

Observability for Performance

Use logs, metrics, traces, and alerts together.

Signal	Useful for
Metrics	trends, saturation, alerts
Logs	detailed events and errors
Traces	request path and latency breakdown
Profiles	CPU, memory, lock, GC bottlenecks
Synthetic checks	external user-visible availability

Visualize Bottlenecks

Performance problems are easier to diagnose when metrics are mapped to architecture.

Useful visualizations:

- Service dependency graph
- Request traces
- Queue depth over time
- p95/p99 latency by endpoint
- CPU/memory/network saturation
- Database slow-query dashboard
- Cost per request by service

Cloud Performance Anti-Patterns

Common anti-patterns:

- Optimizing before measuring
- Scaling app tier but not database tier
- Using average latency only
- Local state in application instances
- Overly chatty service calls
- Missing or stale cache strategy
- Manual deployment changes
- No load testing before launch
- No rollback plan for performance regression

Optimization Playbook

When a cloud application is slow:

1. Confirm the user-visible symptom
2. Check p95/p99 latency and error rate
3. Trace one slow request
4. Identify the bottleneck resource or dependency
5. Decide whether to optimize code, architecture, or capacity
6. Test the change under load
7. Monitor after deployment

Example: API Latency Bottleneck

User request

- Load balancer
- API service
- Auth service
- Database
- Cache
- External payment API

Potential optimizations:

- Cache authentication metadata
- Remove unnecessary synchronous calls
- Add database index
- Use connection pooling
- Add timeout and circuit breaker
- Move slow external call to asynchronous workflow

Example: Queue-Based Worker Bottleneck

Symptoms:

- Queue depth grows
- Worker CPU is low
- p99 processing time increases
- Autoscaler does not add workers

Possible causes:

- Scaling metric is wrong
- External dependency is throttling
- Worker is blocked on I/O
- Batch size is inefficient
- Database writes are serialized

Example: Database Bottleneck

Symptoms:

- Slow queries
- High lock wait time
- High connection count
- CPU or IOPS saturation
- Increased p99 latency

Possible optimizations:

- Add indexes
- Rewrite query
- Add read replicas
- Cache hot reads
- Partition or shard data
- Move analytics away from OLTP database

Example: Network Bottleneck

Symptoms:

- High latency between services
- Cross-region traffic spike
- Large payloads
- High retry rate
- Increased egress cost

Possible optimizations:

- Co-locate dependent services
- Use CDN or edge cache
- Compress or reduce payloads
- Use asynchronous communication
- Avoid synchronous cross-region calls

Summary

Design Checklist

Before deploying a cloud application, ask:

- What are the performance goals?
- What is the expected baseline and peak load?
- What is the autoscaling strategy?
- Where is state stored?
- What is cached?
- What are the critical dependencies?
- What are the p95/p99 targets?
- How will performance regressions be detected?

Operations Checklist

After deployment, continuously check:

- Latency trends
- Error rate trends
- Saturation metrics
- Queue depth
- Slow queries
- Cache hit ratio
- Autoscaling events
- Cost per request
- User feedback
- Performance changes per release

Summary

Cloud application performance optimization combines:

- Requirements and metrics
- Architecture selection
- Compute right-sizing and scaling
- Data-store and query design
- Caching and content delivery
- Network-aware placement
- Modular and stateless application design
- Observability, testing, and continuous review